TECNOLÓGICO NACIONAL DE MÉXICO

INSTITUTO TECNOLÓGICO DE TIJUANA

SUBDIRECCIÓN ACADÉMICA

DEPARTAMENTO DE SISTEMAS Y COMPUTACIÓN

Periodo:

Febrero - Junio 2026

Ingeniería en Sistemas Computacionales

Materia:

Lenguajes de Interfaz

Unidad a Evaluar

I

Tema:

Práctica 4.1

Alumno:

Alfaro Guerra Juan Jesus (23211905)

Maestro: *Rene Solis Reyes*

Aula y Horario:

Grupo / Clave	Aula/Lab.	Hora	Dia/Sem
SC6B	LAB B	17:00 / 18:00	L-J

📄 Proyecto: Optimización con ARM64, C y Python

1. Introducción

¿Qué es ARM64?

ARM64 (también conocido como AArch64) es una arquitectura de 64 bits basada en el conjunto de instrucciones RISC (Reduced Instruction Set Computer). Se caracteriza por su eficiencia energética, alto rendimiento y uso extendido en dispositivos como smartphones, servidores y sistemas embebidos.

¿Por qué usar Assembly?

El lenguaje ensamblador permite interactuar directamente con el hardware, ofreciendo control total sobre el uso de registros, memoria e instrucciones del procesador.

El ensamblador ayuda a entender cómo funciona la computadora a nivel de registros y memoria, permitiendo optimizar código crítico y mejorar significativamente el rendimiento.

2. Marco Teórico

Arquitectura ARM

La arquitectura ARM se basa en principios RISC, lo que implica:

- Instrucciones simples y rápidas
- Bajo consumo de energía
- Alto rendimiento por ciclo de reloj

Registros (x0–x30)

ARM64 cuenta con 31 registros de propósito general:

- x0 – x7: Argumentos de funciones y valores de retorno
- x8: Registro indirecto o auxiliar

- x9 – x15: Temporales
- x16 – x17: Uso interno (intra-procedural call)
- x19 – x28: Registros preservados
- x29: Frame pointer
- x30: Link register (dirección de retorno)

ABI (Application Binary Interface)

El ABI define cómo interactúan las funciones a nivel binario:

- Convención de llamadas
- Uso de registros
- Paso de parámetros
- Manejo de la pila

Esto permite la integración entre lenguajes como C y Assembly.

3. Desarrollo

Explicación de los archivos

🐍 Python

- Actúa como interfaz principal
- Ejecuta pruebas y controla el flujo del programa
- Llama funciones escritas en C

⚙️ C

- Sirve como puente entre Python y Assembly
- Maneja la interoperabilidad mediante funciones exportadas
- Reduce el overhead respecto a Python

🧬 Assembly (ARM64)

- Implementa las rutinas críticas
- Optimiza operaciones a nivel de CPU
- Utiliza registros directamente para máxima eficiencia

4. Resultados

Método	Tiempo
Python	7.677 s
C	7.668 s
ASM (ARM64)	7.657 s

5. Análisis

- El código en Assembly presenta el mejor rendimiento, especialmente en operaciones repetitivas (loops).
- Python introduce un overhead considerable debido a su naturaleza interpretada.
- C ofrece un balance entre rendimiento y facilidad de integración.
- La diferencia de rendimiento se vuelve más evidente en tareas intensivas.

6. Conclusiones

¿Cuándo usar Assembly?

- En secciones críticas de rendimiento
- Cuando se requiere control total del hardware
- En sistemas embebidos o de tiempo real

Ventajas

- Máximo rendimiento
- Control total del hardware
- Optimización precisa

Desventajas

- Mayor complejidad
- Difícil mantenimiento
- Menor portabilidad

7. Evidencias

untitled - asciinema.org

```
  GNU nano 7.2
import ctypes
import time

lib = ctypes.CDLL("./build/libops.so")

lib.add.argtypes = [ctypes.c_long, ctypes.c_long]
lib.add.restype = ctypes.c_long

print("ASM ADD:", lib.add(10, 20))

# Benchmark
def bench():
    start = time.time()
    for _ in range(10_000_000):
        lib.add(1,2)
    end = time.time()
    print("ASM:", end - start)

bench()

File Name to Write: app.py
```

```
  GNU nano 7.2
    ldr x5, [x0, x3, lsl #3]
    ldr x6, [x1, x3, lsl #3]

    mul x7, x5, x6
    add x4, x4, x7

    add x3, x3, #1
    b loop_dot

end_dot:
    mov x0, x4
    ret

File Name to Write: ops.s
```

```
  GNU nano 7.2
CC = gcc
CFLAGS = -fPIC -O2 -Wall -g
LDFLAGS = -shared

TARGET = build/libops.so

all: $(TARGET)

$(TARGET): bridge.o ops.o
        $(CC) $(LDFLAGS) -o $@ $^

bridge.o: bridge.c
        $(CC) $(CFLAGS) -c bridge.c

ops.o: ops.s
        $(CC) $(CFLAGS) -c ops.s

clean:
        rm -f *.o $(TARGET)

File Name to Write: Makefile
```

```
  GNU nano 7.2

long sum_array(long* arr, long n){
    return sum_array_asm(arr,n);
}

long count_even(long* arr, long n){
    return count_even_asm(arr,n);
}

long dot_product(long* a, long* b, long n){
    return dot_product_asm(a,b,n);
}

File Name to Write: bridge.c
```

```
(gdb) set breakpoint pending on
(gdb) break add_asm
Function "add_asm" not defined.
Breakpoint 1 (add_asm) pending.
(gdb) run app.py
Starting program: /usr/bin/python3 app.py
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/aarch64-linux-gnu/libthread_db.so.1".

Breakpoint 1, add_asm () at ops.s:27
27          add x0, x0, x1
(gdb)
```

```
--Type <RET> for more, q to quit, c to continue without paging--q
Quit
(gdb) x/10gx $x0
0xa:    Cannot access memory at address 0xa
(gdb) stepi
28          ret
(gdb) nexti
0x0000ffff7696624 in ?? () from /lib/aarch64-linux-gnu/libffi.so.8
(gdb) exit
A debugging session is active.

        Inferior 1 [process 15139] will be killed.

Quit anyway? (y or n)
```

```
ubuntu@ip-172-31-43-43:~/arm64_project$ gdb python3
GNU gdb (Ubuntu 15.1-1ubuntu1~24.04.1) 15.1
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "aarch64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from python3...
(No debugging symbols found in python3)
```